

Batch: B1 Roll No.:16010122158

Experiment No. 7

Grade: AA / AB / BB / BC / CC / CD / DD

Signature of the Staff In-charge with date

Title: Designing test plan document for Mini Project

Aim: To learn and understand the way of developing the software by classical methods of software engineering.

Planning and monitoring, testing, validating of the project using tools and prepares a document for the same by using the concept of software engineering

CO: 5 Test the given software for different test cases with proper test planning

Books/ Journals/ Websites referred:

1. Roger Pressman, Software Engineering: A practitioners Approach, McGraq Hill, 2010 ,6th edition
 2. Ian Somerville , Software Engineering , Addison Wesley,2011,9th edition
 - 3 http://en.wikipedia.org/wiki/Software_requirements_specification
-

Test Plan Template:

Beverly Automotives

Prepared by:

Asmi Rode , Sakshi Raut, Aakanksh Sen

TABLE OF CONTENTS

1.0 INTRODUCTION

2.0 OBJECTIVES AND TASKS

2.1 Objectives & tasks

3.0 SCOPE

4.0 Testing Strategy

4.1 Alpha Testing (Unit Testing)

4.2 System and Integration Testing

4.3 Performance and Stress Testing

4.4 User Acceptance Testing

4.5 Batch Testing

4.6 Automated Regression Testing

4.7 Beta Testing

5.0 Hardware Requirements

6.0 Environment Requirements

6.1 Main Frame

6.2 Workstation

7.0 Test Schedule

8.0 Control Procedures

9.0 Features to Be Tested

10.0 Features Not to Be Tested

11.0 Resources/Roles & Responsibilities

12.0 Schedules

13.0 Dependencies

14.0 Risks/Assumptions

15.0 Tools

16.0 Approvals

1.0 INTRODUCTION:

The **Beverly Automotives** project is an online car dealership platform aimed at providing users with a seamless experience for browsing, purchasing, and managing cars. This project focuses on creating a clean and responsive user interface, integrating secure payment gateways, and building an efficient inventory management system. Through modern technologies like HTML, CSS, JavaScript, PHP, and SQL, the platform ensures optimal functionality for both users and administrators. This document outlines the project's scope, objectives,

deliverables, and management plan to guide its successful completion.

2.0 2.1 OBJECTIVES AND TASKS:

Objectives:

- Create an interactive car dealership platform, allowing users to browse and purchase cars securely.
- Implement user-friendly navigation, responsive design, and real-time inventory updates.
- Enable smooth online transactions with secure payment integration.
- Ensure scalability for handling growing user and inventory data efficiently.

Tasks:

1. Requirement Gathering: Understand user needs, set goals.
2. UI/UX Design: Create mockups and wireframes.
3. Frontend Development: Build responsive and dynamic pages.
4. Backend Development: Set up database, integrate payment systems.
5. Testing & Debugging: Ensure bug-free functionality.
6. Deployment & Maintenance: Launch and maintain system stability.

3.0 SCOPE:

The Beverly Automotives Website Project aims to develop a fully functional, user-friendly, and responsive platform for car dealerships. The scope includes:

- Frontend: Design of user interfaces to browse car inventory, view detailed car descriptions, and interact with secure online payment options.
- Backend: Implementation of secure authentication, car inventory management, and payment processing.
- Testing & Deployment: Ensuring the website functions smoothly across devices and launching the website for user access.
- Maintenance: Regular updates, bug fixes, and scalability for future growth.

4.0 TESTING STRATEGY:

4.1 Alpha Testing (Unit Testing)

Each component, such as the car listing feature, reservation system, and payment gateway, will be tested individually to ensure they function as intended. This ensures isolated unit functionality, including database connections for the inventory.

4.2 System and Integration Testing

Testing the interaction between front-end modules (UI/UX) and backend features like the search bar, user login, and payment handling. It ensures smooth data flow and cohesive system performance.

4.3 Performance and Stress Testing

Simulating high user traffic or large file uploads (e.g., car images) to test server

response times, database performance, and the site's ability to handle high demand.

4.4 User Acceptance Testing

End-users, such as potential customers and dealership staff, will review the UI/UX and core functions like booking and payments to ensure the system meets customer needs and expectations.

4.5 Batch Testing

Verifying large operations like bulk car uploads, multiple simultaneous user actions, and backend updates to check that the system performs consistently under batch processing.

4.6 Automated Regression Testing

Automating test cases to verify that new features (like advanced search filters or new payment options) do not affect existing functionalities, ensuring a bug-free user experience across updates.

4.7 Beta Testing

A pre-launch test will involve a limited group of users to provide feedback on the user experience and uncover any potential issues. This is done before the official website launch, ensuring high functionality and ease of use for all features.

4.8 *White box testing*

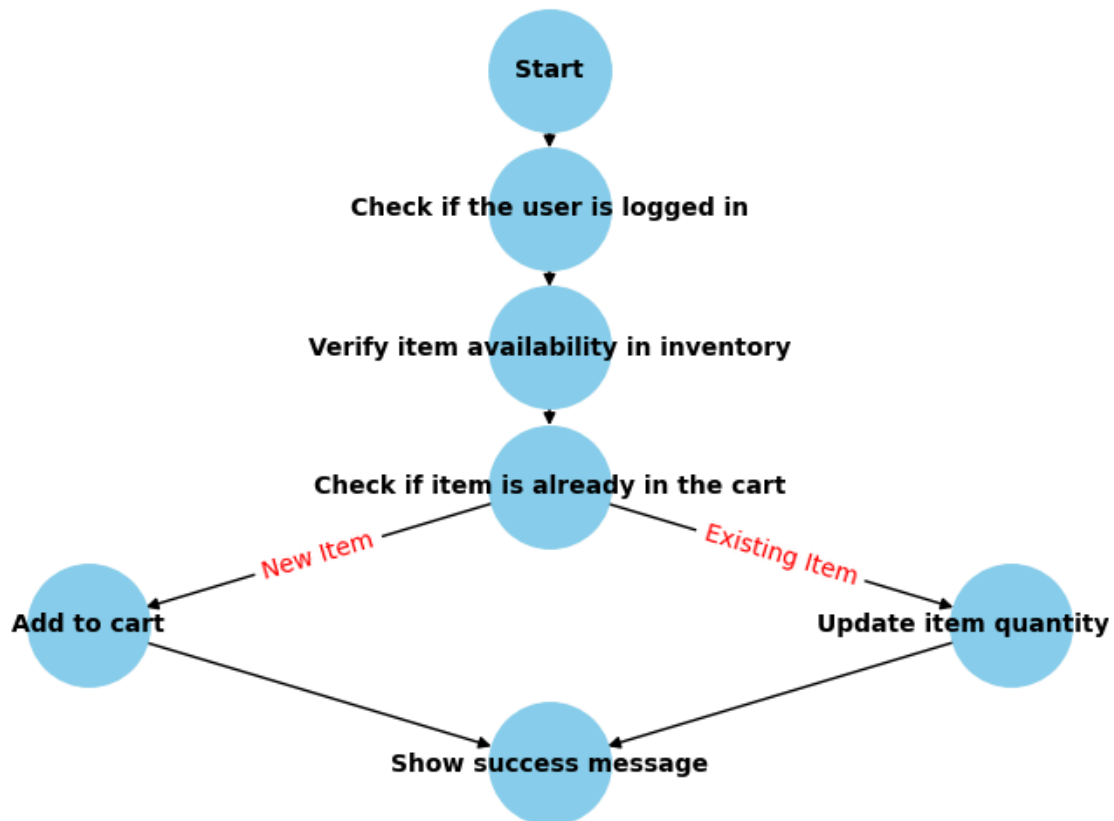
The white box testing strategy for the Beverly Automotive website focuses on ensuring robust, secure, and efficient code across core functionalities. Unit Testing is used to validate individual functions, loops, and conditions to ensure each component works as intended. Data Flow Testing checks the use of variables and input validation to maintain data integrity. Comprehensive Code Coverage is achieved through statement, branch, and path coverage, while Basis Path Testing targets all unique paths in complex functions. API Testing ensures that API responses are accurate and handle errors effectively, enhancing the reliability of backend integrations. Security Testing protects against SQL injection, cross-site scripting (XSS), and ensures secure session management to safeguard user interactions. Resource Leak Testing checks for efficient memory management and database handling, preventing performance degradation. Finally, Code Optimization focuses on refactoring and fine-tuning critical code areas, ensuring the system is responsive and well-optimized. Together, these approaches create a secure, reliable, and optimized codebase.

Create Flow Graph

We'll represent each decision/action involved in adding an item to the cart using nodes and edges. Here's an example breakdown:

1. **Node 1:** Start
2. **Node 2:** Check if the user is logged in
3. **Node 3:** Verify item availability in inventory
4. **Node 4:** Check if item is already in the cart
5. **Node 5:** Add item to cart (if it's a new item)
6. **Node 6:** Update item quantity in the cart (if item already exists)
7. **Node 7:** Show success message

Item Cart Flowchart for E-Commerce System



Calculate Cyclomatic Complexity

Using the formulas provided:

- Regions Count:** The number of regions (areas bounded by edges) in the flow graph.
 - Suppose we have three regions, so $V(G)=3$
- Edges and Nodes Formula:**
 - Edges (E): 7 (1-2, 2-3, 3-4, 4-5, 4-6, 5-7, 6-7)
 - Nodes (N): 7
 - Using $V(G)=E-N+2$: $V(G)=7-7+2=2$
- Predicate Nodes Formula:**
 - Count decision points (predicate nodes) in the flow:
 - Decision points: 2 (for login check and item availability).
 - $V(G)=P+1$: $V(G)=2+1=3$
- Thus, the cyclomatic complexity is 3, indicating three independent paths to cover.

Identify Independent Paths

We can now outline independent paths based on our flow:

- Path 1:** User logged in → Item available → Item not in cart → Add to cart → Success
- Path 2:** User logged in → Item available → Item already in cart → Update quantity → Success
- Path 3:** User not logged in → Redirect to login

Develop Test Cases

Each path will need at least one test case:

1. **Test Case for Path 1 (Happy Path):**
 - **Preconditions:** User is logged in, item is available, item is not in cart.
 - **Action:** Add item to cart.
 - **Expected Result:** Item is added to the cart, quantity set to 1, and a success message is displayed.
2. **Test Case for Path 2 (Existing Item Update):**
 - **Preconditions:** User is logged in, item is available, item is already in the cart.
 - **Action:** Add item to cart.
 - **Expected Result:** Item quantity is updated (e.g., increased by 1), and a success message is displayed.
3. **Test Case for Path 3 (Login Redirect):**
 - **Preconditions:** User is not logged in.
 - **Action:** Attempt to add item to cart.
 - **Expected Result:** User is redirected to the login page, and the item is not added to the cart.

By using this approach, you ensure that the "Add to Cart" functionality has comprehensive test coverage and that all possible execution paths are verified, covering both typical usage scenarios and edge cases.

4.9 AGI

Tax Rules for Beverly Automotive's AGI:

1. If AGI is between \$1 and \$100,000, the tax due is 15% of AGI.
2. If AGI is between \$100,001 and \$500,000, the tax due is 25% of AGI.
3. If AGI is between \$500,001 and \$1 million, the tax due is 30% of AGI.
4. If AGI is above \$1 million, the tax due is 40% of AGI.

Equivalence Classes Based on Conditions:

Condition 1:

$\$1 \leq \text{AGI} \leq \$100,000$

- EC1: $\$1 \leq \text{AGI} \leq \$100,000$ (Valid input, taxed at 15%)
- EC2: $\text{AGI} < 1$ (Invalid input)

Condition 2:

$\$100,001 \leq \text{AGI} \leq \$500,000$

- EC3: $\$100,001 \leq \text{AGI} \leq \$500,000$ (Valid input, taxed at 25%)

Condition 3:

$\$500,001 \leq \text{AGI} \leq \1 million

- EC4: $\$500,001 \leq \text{AGI} \leq \1 million (Valid input, taxed at 30%)

Condition 4:

$\text{AGI} > \$1 \text{ million}$

- EC5: $\text{AGI} > \$1 \text{ million}$ (Valid input, taxed at 40%)

Invalid Range:

- EC6: $\text{AGI} < 0$ (Invalid input, as income cannot be negative)

Summary of Equivalence Classes:

Condition	Equivalence Class	Range	Valid/Invalid	Tax Rate
Condition 1	EC1	$\$1 \leq \text{AGI} \leq \$100,000$	Valid	15%
Condition 1	EC2	$\text{AGI} < 1$	Invalid	-
Condition 2	EC3	$\$100,001 \leq \text{AGI} \leq \$500,000$	Valid	25%
Condition 3	EC4	$\$500,001 \leq \text{AGI} \leq \1 million	Valid	30%
Condition 4	EC5	$\text{AGI} > \$1 \text{ million}$	Valid	40%
Invalid Range	EC6	$\text{AGI} < 0$	Invalid	-

Condition 1	EC1	$\$1 \leq AGI \leq \$100,000$	Valid	15%
	EC2	$AGI < 1$	Invalid	N/A
Condition 2	EC3	$\$100,001 \leq AGI \leq \$500,000$	Valid	25%
Condition 3	EC4	$\$500,001 \leq AGI \leq \1 million	Valid	30%
Condition 4	EC5	$AGI > \$1 \text{ million}$	Valid	40%
Invalid Range	EC6	$AGI < 0$	Invalid	N/A

5.0 HARDWARE REQUIREMENTS:

- Device which can open browsers like Chrome
- Internet connectivity

6.0 ENVIRONMENT REQUIREMENTS:

6.1 Main Frame

- Browser(Chrome, Mozilla, Microsoft Edge, etc)
- PostgreSQL server
- Vs code editor
- Github
- Django framework
- Website hosting services like heroku
- Internet Connection
- User PC

6.2 Workstation

The website would be deployed using a hosting provider. Users can use the website from their PCs or mobiles with internet connectivity.

7.0 TEST SCHEDULE:

Test	Responsibility	Start Date	End Date
(A) Unit Testing			

User Login and Registration	Verify that users can register and login with valid information, handle errors when incorrect credentials are provided. Test reset password functionality. Check if user account details are displayed correctly from the database.	16/11/2024	18/11/2024
User Account	Test if user can update account details Verify whether users can track their order history.	16/11/2024	18/11/2024
Car Browsing	Verify that the search function returns relevant results. Check that product image, description and ratings load correctly from the database. Test sorting and filtering options.	19/11/2024	25/11/2024
Checkout and Payment	Check if the checkout function works smoothly and as intended Verify that the payment gateways are properly integrated. Ensure that users receive appropriate notifications for order confirmation.	06/12/2024	20/12/2024
(B) System and Integration Testing			

End-to-End testing	Testing the entire user journey, from logging in to making a purchase	21/12/2024	31/12/2024
Data Integrity Testing	Verify if data is correctly stored and retrieved from database	01/01/2024	03/01/2024
Compatibility Testing	Test whether the website is compatible on different browsers	04/01/2024	10/12/2024
(C) Performance and Stress Testing			
Load testing	Test how many users can simultaneously access the website without degrading its performance	11/01/2024	15/01/2024
Scalability testing	Test the website's ability to handle increased loads by adding more resources (servers).	16/01/2024	20/01/2024
Response time testing	Measure time taken for website to respond to user actions. Check whether the measured time falls within the acceptable response time range. Assess load times for various pages.	21/01/2024	25/01/2024
(D) User Acceptance Testing			
Accessibility testing	Test how accessible the website is to users with disabilities	26/01/2024	30/01/2024

Feedback and customer service	Document user issues gathered from feedback forms for future tests and fixes.	01/02/2024	---
(E) Regression Testing			
Regression testing	Check if new fixes cause any other bugs or problems in other modules.	01/02/2024	---

8.0 CONTROL PROCEDURES:

Problem Reporting

A **Bug Report in Software Testing** is a detailed document about bugs found in the software application. Bug report contains details about bugs like description, date when bug was found, name of tester who found it, name of developer who fixed it, etc. Bug report helps to identify similar bugs in future so it can be avoided.

While reporting the bug to developer, your Bug Report should contain the following information namely, Defect_ID, Defect Description, Version, Steps, Date Raised, Reference, Detected By, Status, Fixed by, Date Closed, Severity, Priority

Change Requests

The developer who encountered the bug would report to the manager and also start working on the bug. After it is fixed, another report containing details like time, date solved, description of the fix, complexity of the fix etc shall be sent to the head developer and he shall approve the changes.

9.0 FEATURES TO BE TESTED:

Registering as new users with required authentication (if applicable).

Logging in to the website.

Homepage with working buttons, navigation, and search bar.

User Profile Page allowing edits to user information.

Viewing Car Listings with accurate photos and details displayed.

Searching for Cars based on make, model, and other keywords.

Applying Filters for refining car searches (e.g., by price, year).

Car Details Page showing full car specs and availability.

Submitting Enquiry Forms for bookings or requests.

Logging Out securely.

Error Messages for invalid inputs or actions.

10.0 FEATURES NOT TO BE TESTED:

Static pages like the 'About Us' page do not need testing. Features which are to be removed from the website in further versions also shouldn't be tested.

11.0 RESOURCES/ROLES & RESPONSIBILITIES:

Specify the staff members who are involved in the test project and what their roles are going to be (for example, Mary Brown (User) compile Test Cases for Acceptance Testing). Identify groups responsible for managing, designing, preparing, executing, and resolving the test activities as well as related issues. Also identify groups responsible for providing the test environment. These groups may include developers, testers, operations staff, testing services, etc.

12.0 SCHEDULES:

Major Deliverables

- SRS document
- Test Plan
- Test Cases
- Test Incident Reports
- Test Summary Reports

13.0 DEPENDENCIES:

Since there are many modules on the website, time will be a major constraint to carry out testing for each and every module.

14.0 RISKS/ASSUMPTIONS:

- Programme Manager will be responsible for giving a timely sign-off on the deliverables, test data covering all the testing scenarios, procuring any tools' licenses required for the successful facilitating testing activities.
- Functionalities are delivered as per schedule for testing.
- It is assumed that all the Signed off Functional Specification documents requisite test environments/applications with product parameters setup done and interfaces will be provided on start of Testing execution i.e. The complete build is expected to be available on day one of testing

- Planning estimates include test case designs/Testwares for CRs & modification of Regression packs as per the scope. Estimates include cost of one review and one rework based on the review comments of the same by BA & TO.
- Code is fully Unit tested and the closure report to be shared by the development team before handing over code to the testing team.
- Any impact to testing budget due to delay in Build, Environment, Defect fix, interface availability will be handled through the Change Request process. Changes if any to the requirement / functional specifications documents during SIT will follow the change request process needing separate budget and estimate.
- Application and environment should be stable. There should be no downtime or error in functionality. Environments, connectivity, User access (Testers), IP opening, Firewall opening to be taken care of by the Infrastructure team.

15.0 TOOLS:

Testing was done using Vs code where bugs are reported in the terminal itself.

16.0 APPROVALS:

Conclusion:

We were able to document the testing phase of software engineering. We came to know about the procedures, different reports, different roles and responsibilities that are encountered in the testing phase of a software.

Post Lab Descriptive Questions:

1. Distinguish between Black Box and White Box Testing

Black Box Testing	White Box Testing
It is a way of software testing in which the internal structure or the program or the code is hidden and nothing is known about it.	It is a way of testing the software in which the tester has knowledge about the internal structure or the code or the program of the software.
Implementation of code is not needed for black box testing.	Code implementation is necessary for white box testing.
It is mostly done by software testers.	It is mostly done by software developers.
No knowledge of implementation is needed.	Knowledge of implementation is required.
It can be referred to as outer or external software testing.	It is the inner or the internal software testing.
It is a functional test of the software.	It is a structural test of the software.
This testing can be initiated based on the requirement specifications document.	This type of testing of software is started after a detail design document.
No knowledge of programming is required.	It is mandatory to have knowledge of programming.
It is the behavior testing of the software.	It is the logic testing of the software.
It is applicable to the higher levels of testing of software.	It is generally applicable to the lower levels of software testing.
It is also called closed testing.	It is also called as clear box testing.
It is least time consuming.	It is most time consuming.
It is not suitable or preferred for algorithm testing.	It is suitable for algorithm testing.

Black Box Testing	White Box Testing
Can be done by trial and error ways and methods.	Data domains along with inner or internal boundaries can be better tested.
Example: Search something on google by using keywords	Example: By input to check and verify loops
Black-box test design techniques- • Decision table testing • All-pairs testing • Equivalence partitioning • Error guessing	White-box test design techniques- • Control flow testing • Data flow testing • Branch testing
Types of Black Box Testing: • Functional Testing • Non-functional testing • Regression Testing	Types of White Box Testing: • Path Testing • Loop Testing • Condition testing
It is less exhaustive as compared to white box testing.	It is comparatively more exhaustive than black box testing.

2. Explain traceability matrix.

A **traceability matrix** is a document that maps and traces the relationship between two baseline documents, often **requirements** and **test cases**. It's used to ensure that all requirements are covered by test cases and helps track project progress, assess test coverage, and identify gaps in testing.

Key Components:

1. **Requirement ID:** A unique identifier for each requirement.
2. **Requirement Description:** Brief description of the requirement.
3. **Test Case ID:** The identifier of each test case that tests a specific requirement.
4. **Test Case Description:** Brief summary of the test case.

Types of Traceability Matrix:

1. **Forward Traceability:** Links requirements to test cases to ensure all requirements are covered by tests.
2. **Backward Traceability:** Links test cases back to requirements to confirm each test aligns with a requirement.
3. **Bi-Directional Traceability:** Combines both forward and backward traceability for comprehensive coverage.

Example of a Traceability Matrix:

Requirement ID	Requirement Description	Test Case ID	Test Case Description
RQ-01	User login functionality	TC-01	Verify successful user login
RQ-02	Password reset feature	TC-02	Verify password reset email
RQ-03	Product search feature	TC-03, TC-04	Verify search and filter options

Benefits:

- **Ensures Test Coverage:** Confirms every requirement is tested.
- **Helps in Impact Analysis:** Tracks the impact of requirement changes.
- **Improves Quality:** Reduces the risk of missing requirements in testing.
- **Streamlines Project Tracking:** Offers a structured way to track testing progress and identify gaps.